Übungsblatt 7: Tracing und Timing-Analyse

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Ziele und Ablauf

Sie definieren Messintervalle, rechnen Zeitgrößen und beurteilen Stichproben und Aufzeichnungslücken. Grundlage sind Vorlesungen 1–7. Ein Trace ist eine geordnete Aufzeichnung ausgewählter Ereignisse. Ein Messvertrag benennt Anfang, Ende, Quelle, Einheit und Bedingungen.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	32 Orientierungspunkte
Transfer und Besprechung	15	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **80 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 80 Minuten Erklärung und 20 Minuten Quiz/Abschluss zum Termin mit 180 Minuten.

Teil A: Papieraufgaben

Aufgabe 1: Ausgabe und Beobachtereffekt · 8 Punkte

Eine Funktion erledigt in einem vereinfachten Modell 10 µs eigene Arbeit. Danach gibt sie die fünf ASCII-Zeichen `Hallo` bei 115200 bit/s über UART aus: ein Startbit, acht Datenbits, keine Parität, ein Stopbit je Byte. Kein Nullterminator oder Zeilenumbruch wird übertragen. Relative Abschlussfrist ab Funktionsbeginn: 50 µs.

1. Berechnen Sie Leitungsbitzahl und reine Leitungsdauer. (2 P)
2. Im **blockierenden Lehrfall** liegt die volle Leitungsdauer zusätzlich innerhalb des Antwortintervalls, weitere Kosten sind null. Berechnen Sie Antwortzeit und Fristabweichung. (2 P)
3. Warum wird dieselbe Leitungsdauer bei IRQ- oder DMA-Ausgabe nicht automatisch zur gleichen CPU-Zeit? Welche Kosten oder Wartephasen können trotzdem verbleiben? (2 P)
4. Definieren Sie einen GPIO-Messvertrag um die ursprüngliche Funktion. Nennen Sie zwei Gründe, warum ein längerer Puls keine reine Funktions-CPU-Zeit belegt, und eine geeignete Position für die Ausgabe. (2 P)

Aufgabe 2: Jobspur und Interruptkette · 8 Punkte

Jobmodell: A wird bei 1 ms freigegeben, läuft 2–4 ms und 6–7 ms und ist bei 7 ms fertig. Von 4–6 ms erledigt der Prozessor fremde Arbeit. Absolute Deadline: 8 ms. Kontextwechselkosten sind in diesem Modell null.

1. Berechnen Sie eigene CPU-Zeit, Antwortzeit, Startwartezeit und Slack. Wird die Frist gehalten? Warum ist „fertig“ nicht automatisch das Ende der Tasklebensdauer? (4 P)

Interruptmodell, gemeinsame hypothetische Uhr:

Ereignis	Zeit in µs
externe Flanke	100
ISR-Eintritt	106
Veröffentlichung	110
Task Ready	111
ISR-Ende	115
Taskstart	120
geforderte Reaktion	128

2. Berechnen Sie Eintrittslatenz, ISR-Intervall, Ready-Wartezeit und Gesamtreaktion. Geben Sie eine disjunkte Zerlegung der Gesamtreaktion an und begründen Sie, warum die vier Kennzahlen nicht miteinander aufsummiert werden. (4 P)

Aufgabe 3: Lokaler Zähler und Aussagegrenze · 8 Punkte

DWT-CYCCNT ist hier ein vorzeichenloser 32-Bit-Zähler eines Kerns. Seine Unterstützung und sein Lauf seien für das Modell bestätigt.

1. Anfang `0xFFFFFFF0`, Ende `0x00000030`: Berechnen Sie die vorzeichenlose Modulo-Differenz. Bei hypothetisch stabilen 400 MHz: Welche Zeit entspricht ihr und wie lang ist ein voller Umlauf? Warum genügt die Differenz allein nicht für ein beliebig langes Intervall? (3 P)

2. Tausend Takte werden einmal bei 400 MHz und einmal bei 200 MHz gezählt. Rechnen Sie beide Zeiten. Darf man den Endstand des M4 vom Anfangsstand des M7 subtrahieren? Welche Voraussetzungen fehlen? (2 P)
3. Eine Stichprobe vollständig definierter Intervalle enthält 12, 15, 14 und 18 µs. Berechnen Sie Anzahl, Minimum, beobachtetes Maximum und Mittelwert. Warum ist das Maximum keine WCET? Warum kann ein CYCCNT-Fenster um eine Funktion auch fremde Arbeit enthalten? (3 P)

Aufgabe 4: Quelle, Kapazität und Datenqualität · 8 Punkte

1. Ein konfiguriertes DWT-Ereignis zeigt einen M7-Store mit eins. Der Speicherbereich ist tatsächlich Write-Back-cachebar, M7 hat eine Dirty-Kopie, SRAM kann noch null enthalten. Beweist das Ereignis den vom M4 gelesenen Wert? Was muss bei einem Filter auf `ipc.payload` zusätzlich geprüft werden? Das Layout aus Vorlesung 6 hat Offsets 0/4/8. (2 P)
2. Während eines M4-Interrupts zeigt `pxCurrentTCB` weiter auf Task A. Läuft damit A-Code? Unterscheiden Sie Taskliste, Running-Profil und Zustandshistorie. (2 P)
3. Synthetisches Ratenmodell: 250000 Ereignisse/s mit vier Byte je Ereignis, Abfluss 600000 Byte/s, anfangs freier Puffer 64 KiB. Kein Paketoverhead, konstante Raten. Wann ist der Puffer voll? Würde doppelte Kapazität die dauerhafte Überlast lösen? (2 P)
4. Im Trace fehlt ein Anfangspaket. Später enthält ein Abschnitt eine Verlustmarke und danach bestätigte neue Synchronisation mit passendem Kontext. Darf der Anfang als null gezeichnet und die Lücke interpoliert werden? Wie behandeln Sie den späteren vollständigen Abschnitt? (2 P)

Transfer: Was widerlegt eine Garantie? · 15 Minuten

Bearbeiten Sie zehn Minuten und vergleichen Sie fünf Minuten. Keine zusätzlichen Punkte. Vier vollständige Jobantwortintervalle aus einer gemeinsamen **Modelluhr** betragen 38, 42, 55 und 41 µs. Die relative Deadline ist 50 µs. Anfänge und Abschlüsse sind vollständig erfasst; es gibt in diesem Abschnitt keine Lücke.

1. Berechnen Sie Mittelwert und Maximum. Ist die Frist in allen vier Fällen erfüllt?
2. Ein Review meint: „Die Stichprobe beweist keine WCET, deshalb zählt der 55-µs-Fall nicht als Widerlegung.“ Beurteilen Sie das Argument. Unterscheiden Sie dabei Antwortzeit von eigener CPU-Zeit.
3. In einem anderen Trace fehlt der Abschluss des Jobs. Wie wird dieser Fall beurteilt, ohne ihm null oder einen erfundenen Höchstwert zuzuweisen?

Teil B: Praxisfall · 15 Minuten

Praxisfall 7: Ein Zählerfenster auswerten

Papieralternative, synthetische Zählerdaten: Ein bestätigter lokaler Zähler läuft bei konstant 200 MHz, ohne Halt oder Umlaufmehrdeutigkeit. Jeder Eintrag umfasst die Zeit zwischen zwei Zählerlesungen, einschließlich Barrieren und möglicher Interrupts. Logging liegt außerhalb. Drei EMPTY-Fenster ergeben 200, 220 und 240 Takte; drei WORK-Fenster 2000, 2400 und 2200 Takte.

1. Berechnen Sie Minimum, Maximum und Mittelwert je Gruppe in µs sowie die WORK-Spanne.
2. Ein Review zieht von jedem Arbeitswert das kleinste Leerfenster ab und nennt den größten Rest „WCET der Funktion“. Welche zwei Nachweislücken bleiben?
3. Welche Angaben brauchen Sie zusätzlich, wenn ähnliche Zahlen aus einem echten Boardlauf stammen?

Bei vorbereitetem Board: Die Lehrperson zeigt einen abgeschlossenen `lab07_timing`-Lauf und die beiden Gruppen. Der reale Baustein hat sieben Einträge je Gruppe; die drei Papierwerte sind ein unabhängiges Lehrmodell. Neue Tracehardware und ein BlueBox-/winIDEA-Aufbau sind freiwillige Vertiefung.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (32 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 7; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*, 3. Auflage; ST RM0399, UM2408 und AN4989; tatsächliche CMSIS-M7-Header, insbesondere DWT-Register und ITM_SendChar; Dokumentation der installierten STM32 VS Code Extension und gegebenenfalls des vorhandenen winIDEA-/BlueBox-Aufbaus.